Handwritten Explanation

Bug 1: SQL Operator Precedence

TaskRepository.java line 14-16, also in search_task.sql & task_search_package.sql

Layer: Backend / SQL

How I found it: Claude Code flagged it. I read the query and understand the issue.

Root Cause: SQL evaluates AND before OR. So the query was being read wrong by the database. Archived (deleted) task were showing up in results when their description matched a search. Also the status filter was never applied to title matches.

Fix: Wrapped the title / description parts in brackets so AND and OR are unambiguous. Same fix applied in all 3 files.

Bug 2:- UI stuck in loading forever File: useTasks.js
line 19-21 Layer: Frontend

How I found it:- claude code pointed to the catch block. I read it & saw loading was never turned off on error.

Root cause:- setLoading(false) was only called on success, not on failure. So if the API call failed, the screen showed "Loading tasks" forever.

Fix. - Added setLoading(false) inside the catch block.

after retry File

Bug 3: Old error messages shown after retry File useTasks.js Layer: Frontend

How I found it: Claude Code flagged it. I read & saw error state was never cleared at the start.

Root cause: ~~setErr~~ setError(null) was never called when a new fetch began. So if the first failed the error message stayed on the screen even after a ~~successful~~ successful retry.

Fix - Added setError(null) at the top of the useEffect, before the fetch starts.

Bug 4: Wrong Page shown after filter change

File: App.jsx line 24–25 layer: Frontend

How I found – claude code flagged it. I traced how setQuery & setStatus were used

Root Cause: When user changed the search or status filter, the page number stayed whenever it was. So if you were on page 3 & changed the filter, it tried to show page 3 of the new results – which is often empty

Fix – wrapped both setters to also call setPage(1) so any filter change resets to page 1.

Bug 5 – Race condition on fast typing Files:

Bug 5 - Race condition on fast typing Files:
useTasks.js, api.js. Layer ~~Fr~~ Frontend.

How I found it:- Claude code, got it.

Root cause:- If a user ~~typically~~ typed quickly multiple API called fired. A slow older response could come back after a faster newer one, replacing correct data with stale data.

Fix:- added an AbortController. Each fetch get a signal, and when the effect re-runs, the old request is aborted. AbortError is silently ignored.

BUG 6:- Artificial delay in backend File:- TaskController.java line 36-42 Layer: Backend

How I found it - Claude code flagged it. I read the code & saw Thread.sleep with a label saying "query complexity estimation".

Root Cause:- The code slept for up to 1000ms on every request, worse for short or empty queues. The complexityScore variable was printed but never used for anything real. It was just fake latency.

Fix - Removed the entire sleep block & cleaned up the println that referenced the now-deleted variable.

Bug 7 Bad status input crashes with 500 error File: TaskController.java line: 32

Layer: Backend

How I found it - Claude got flagged it.

Root cause:- If someone sent ?status=invalid TaskStatus.valueOf() throws an IllegalArgumentException. Spring caught this & returned a 500 (server error) instead of a 400 (bad request from user)

Fix - Wrapped in try/catch block.

~~Bug~~

Bug 8: App crashes on null status File:- TaskTable.jsx line 34 · Layer: Frontend

How I found it: Claude code flagged it.

Root cause: task.status.toLowerCase() throws a TypeError if status is null. This would crash the entire table, not just one row.

Fix:- Changed to task.status ?.toLowerCase(). Optional chaining means it returns undefined instead of crashing if status is null.

Bug 9 Console.log left in production code file; api.js line 11 logs frontend

How I found it: I saw it in the code.

Root cause: It can cause non-security issue. It shows the URL in the production & It can ~~contain~~ contain query parameters that can reveal internal behavior.

Fix: Deleted the console.log.